ERR0RS ULTIMATE — Complete Function & Command Reference

```
ERR0RS ULTIMATE v3.0 — Full System Reference
Author: Gary Holden Schneider (Eros) | GitHub: Gnosisone
Root: H:\ERR0RS-Ultimate\
```

TABLE OF CONTENTS

1. HTTP API Endpoints
2. WebSocket Protocol
3. Shell Utility Functions
4. Command Router (`route_command`)
5. Tool Executor — Command Builders
6. Tool Executor — Output Parsers
7. ToolExecutor Class Methods
8. Live Terminal Engine
9. Intent Parser
10. Inline Education Annotator
11. LiveProcess Class Methods
12. Teach Engine — Knowledge Base Topics
13. Natural Language Interface
14. Agent Orchestrator
15. SOC Blue Team Commands
16. Pentest Tool Quick Reference
17. Payload Studio Validators

---

1. HTTP API ENDPOINTS

All endpoints served at `http://127.0.0.1:8765`

Method	Endpoint	Description
GET	`/`	Main dashboard UI (index.html)
GET	`/api/status`	System status: Ollama, MCP, live terminal, WebSocket
GET	`/api/phases`	Pentest phase progress (1–6)
GET	`/api/tools`	List all tools and availability check
GET	`/api/ws_info`	WebSocket URL + availability flags
GET	`/api/payload_studio/snippets`	DuckyScript snippet library
POST	`/api/command`	Route a text command through ERR0RS brain
POST	`/api/tool`	Run a specific tool with target + params
POST	`/api/shell`	Execute raw shell command
POST	`/api/teach`	Request a lesson on any tool/concept
POST	`/api/ollama`	Send prompt to local Ollama model
POST	`/api/rocketgod`	RocketGod RF hardware toolbox
POST	`/api/badusb`	BadUSB / DuckyScript generator
POST	`/api/soc`	SOC blue team one-click security actions
POST	`/api/payload_studio/explain`	Explain a DuckyScript line
POST	`/api/payload_studio/suggest`	Get next-line suggestions
POST	`/api/payload_studio/validate`	Validate DuckyScript syntax

---

2. WEBSOCKET PROTOCOL

Server: `ws://127.0.0.1:8766`

Client → Server Messages

Type	Payload	Description
`run`	`{"command": "nmap 192.168.1.1", "teach": true}`	Run a command / tool
`stdin`	`{"data": "yes\n"}`	Send input to running process
`key`	`{"data": "\x03"}`	Send raw keypress (e.g. Ctrl+C)
`kill`	`{}`	Terminate current running process

Server → Client Messages

Type	Payload	Description
`system`	`{"data": "[ERR0RS] ..."}`	System/status messages
`output`	`{"data": "PORT 22 open"}`	Live tool output lines
`teach`	`{"data": "[ERR0RS] Port 22 = SSH..."}`	Inline education annotation
`teach_block`	`{"data": "=====\n[ERR0RS TEACHES]..."}`	Full lesson block
`error`	`{"data": "PTY fork failed"}`	Error messages
`done`	`{"data": "[ERR0RS] Process complete."}`	Process finished

---

3. SHELL UTILITY FUNCTIONS

`run_shell(cmd, timeout=60) → dict`

Execute a shell command. Returns `{stdout, stderr, returncode, command, status}`. Status values: `"success"`, `"error"`, `"timeout"`

`run_tool_async(tool, target, params) → dict`

Run a registered tool via ToolExecutor in a new asyncio loop. Returns full ToolResult as dict: `{tool, command, status, stdout, stderr, returncode, duration_ms, findings, error}`

`query_ollama(prompt) → dict`

Send a prompt to the local Ollama API (`localhost:11434`). Returns `{stdout, status, source:"ollama"}`. Falls back gracefully if offline.

---

4. COMMAND ROUTER — `route_command(cmd)`

Routes plain English or command text to the right handler.

Trigger	Handler
"teach me", "how do i", "explain", "what is", "learn"	`handle_teach_request()`
Bare tool name (e.g. "sqlmap")	`handle_teach_request()`
`tool target` format (e.g. "nmap 192.168.1.1")	`run_shell()` or `run_tool_async()`
"status", "sysinfo", "whoami"	`run_shell("uname -a && whoami && ip addr")`
"report", "debrief"	`run_shell("python3 demo_report.py")`
"mcp", "kali tools"	`run_shell("mcp-server --list-tools")`
"ollama", "models", "ai status"	`run_shell("ollama list")`
`$cmd` or `!cmd` prefix	Raw shell passthrough
RocketGod keywords	`handle_rocketgod_request()`
BadUSB / Flipper / DuckyScript	`nlp_to_flipper()`
Unknown input	`query_ollama()` fallback

Tool map (verb → tool):

Verb	Tool
nmap, scan	nmap
nikto	nikto
gobuster, dirbust	gobuster

sqlmap, sqli	sqlmap
hydra, brute	hydra
subfinder, subdomain	subfinder
nuclei	nuclei
amass	amass
metasploit, msf	metasploit
hashcat, crack	hashcat

---

5. TOOL EXECUTOR — COMMAND BUILDERS

Each builder takes `(target: str, params: dict) → str`

`build_nmap(target, params)`

`nmap -sV -sC --top-ports 1000 -T3 -oA /tmp/errorz_nmap <target>` Params: `ports` ("top-1000"|"1-65535"|custom), `timing` (1-5), `os_detection`, `version_detection`

`build_sqlmap(target, params)`

`sqlmap -u http://<target> --batch --level=3 --risk=2` Params: `dbs` (bool), `dump` (bool)

`build_nikto(target, params)`

`nikto -h <target> -o /tmp/errorz_nikto.html -format html`

`build_gobuster(target, params)`

`gobuster dir -u http://<target> -w <wordlist> -t <threads>` Params: `wordlist`, `threads`

`build_hydra(target, params)`

`hydra -l <user> -P <wordlist> <target> <service>` Params: `service`, `wordlist`, `username`

`build_hashcat(target, params)`

`hashcat -m <mode> -a 0 <target> <wordlist>` Params: `hash_mode`, `wordlist`

`build_subfinder(target, params)`

`subfinder -d <target> -o /tmp/errorz_subfinder.txt -json`

`build_nuclei(target, params)`

`nuclei -t http://<target> -json -o /tmp/errorz_nuclei.json`

`build_amass(target, params)`

`amass enum -d <target> -o /tmp/errorz_amass.txt -json`

`build_metasploit(target, params)`

Writes `/tmp/errorz_msf_resource.rc` then runs `msfconsole -r <rc> -q` Params: `module`, `options` (dict)

`build_generic(tool_name, target, params)`

Fallback: `<tool> <target> [extra_flags]` Params: `extra_flags` (list)

`_sanitise(value)`

Internal. Strips all characters except alphanumeric, `.`, `-`, `:`, `/`, `_`

---

6. TOOL EXECUTOR — OUTPUT PARSERS

Each parser takes `stdout: str → List[dict]`

`parse_nmap_output(stdout)`

Extracts: `{port, protocol, state, service}` per open port line.

`parse_subfinder_output(stdout)`

Extracts: `{subdomain}` per line.

`parse_gobuster_output(stdout)`

Extracts: `{path, status_code}` from `Found: /path (Status: 200)` lines.

`parse_nuclei_output(stdout)`

Parses JSON lines. Extracts: `{template, severity, matched}`.

`parse_generic_output(stdout)`

Fallback. Extracts: `{raw: line}` for every non-empty line.

---

7. TOOLEXECUTOR CLASS METHODS

Class: `src/core/tool_executor.py — ToolExecutor`

`__init__(on_line=None)`

Optional async callback `on_line(tool_name, line)` for live streaming.

`async run(tool_name, target, params=None) → ToolResult`

Main entry. Finds binary, builds command, executes, parses output. Returns `ToolResult(tool_name, command, status, stdout, stderr, return_code, duration_ms, findings, error)`

`async _execute(tool_name, cmd_str, timeout) → ToolResult`

Internal. Spawns subprocess, streams stdout/stderr, handles timeout.

`async run_batch(tasks, parallel=False) → List[ToolResult]`

Run multiple tools. `tasks = [{"tool":"nmap","target":"x.x.x.x","params":{}}]` `parallel=True` uses `asyncio.gather()`.

Default Timeouts:

Tool	Seconds
nmap	300
sqlmap	600
nikto	180
gobuster	600
hydra	600
hashcat	1800
subfinder	120
nuclei	300
amass	300
metasploit	600

---

8. LIVE TERMINAL ENGINE

File: `src/core/live_terminal.py`

`parse_intent(raw) → dict`

Parses natural language into structured intent. Returns: `{tool, target, raw_cmd, execute, teach, teach_topic, mode}` Modes: `"execute"` | `"teach"` | `"both"` | `"interactive"`

Detected tools: nmap, sqlmap, nikto, gobuster, hydra, metasploit, hashcat, nuclei, subfinder, aircrack, wpscan, enum4linux, ffuf, wfuzz, crackmapexec

`build_command(tool, target) → str`

Builds sensible default command string for tool + target.

Built-in command templates:

Tool	Default Command
nmap	`nmap -sV -sC --top-ports 1000 <target>`
sqlmap	`sqlmap -u '<target>' --batch --dbs --level=1 --risk=1`
nikto	`nikto -h <target>`
gobuster	`gobuster dir -u http://<target> -w common.txt -t 20`
hydra	`hydra -l admin -P rockyou.txt <target> ssh -t 4`
metasploit	`msfconsole -q`
hashcat	`hashcat -m 0 -a 0 <target> rockyou.txt`
nuclei	`nuclei -u http://<target> -s critical,high,medium`
subfinder	`subfinder -d <target> -silent`
aircrack	`aircrack-ng <target>`
wpscan	`wpscan --url http://<target> --enumerate p,u`
enum4linux	`enum4linux -a <target>`
ffuf	`ffuf -u http://<target>/FUZZ -w common.txt`
wfuzz	`wfuzz -w common.txt --hc 404 http://<target>/FUZZ`
crackmapexec	`crackmapexec smb <target>`

---

9. INTENT PARSER — TEACH TRIGGERS

Words/phrases that activate teach mode:

`teach me` · `teach` · `and teach` · `explain` · `what does` · `help me understand` · `why` · `how does` · `what is` · `show me` · `--teach` · `-t` · `and explain` · `walk me through` · `break it down` · `i want to learn` · `educate me` · `what are`

Target extraction regex: Detects IPv4, CIDR, hostnames, and URLs after keywords: `on | against | target | scan | at | for`

---

10. INLINE EDUCATION ANNOTATOR — `annotate_line(line, tool, target)`

Returns list of output events `[{type, data}]` with optional `teach` events.

Nmap Annotations

- **Open port** → Looks up port in `PORT_LESSONS` dict → emits teaching card
- **CVE found** → Links to NVD + Metasploit search suggestion
- **VULNERABLE** → Step-by-step documentation checklist
- **OS detected** → Narrows exploit search to platform

SQLMap Annotations

- "is vulnerable" or "injection found" → Full exploitation workflow
- "available databases" → Next steps for database enumeration
- "password hash" → Hashcat cracking commands

Gobuster Annotations

- **200 OK** → Visit in browser + investigation tips
- **301/302 Redirect** → Follow redirect advice
- **403 Forbidden** → Bypass technique suggestions

Hydra Annotations

- "login: / password:" → Credential reuse checklist

Nikto Annotations

- OSVDB / "+" findings → Research + Metasploit cross-reference tip

`PORT_LESSONS` Dictionary

Ports with built-in lessons: `21, 22, 23, 25, 53, 80, 88, 110, 135, 139, 143, 389, 443, 445, 1433, 1521, 3306, 3389, 5432, 5900, 6379, 8080, 8443, 27017`

Each entry: `(title, attack_tip_with_{target}_placeholder)`

---

11. LIVEPROCESS CLASS METHODS

Class: `src/core/live_terminal.py — LiveProcess`

`__init__(command, tool, target, output_callback, teach_mode)`

Constructor. Sets up PTY process params.

`start() → bool`

Forks PTY subprocess. Starts `_read_loop` thread. Emits system event.

`_emit_pre_teach()`

Internal. If teach_mode=True, calls `handle_teach_request(tool)` before execution starts.

`_read_loop()`

Internal thread. Reads PTY fd with `select.select()`, decodes UTF-8, processes lines. Emits `{"type":"system","data":"[ERR0RS] Process complete."}` on exit.

`_process_line(line)`

Routes line through `annotate_line()` in teach mode or emits plain output event.

`send_input(text)`

Write `text + "\n"` to PTY file descriptor (stdin to running process).

`send_raw(data: bytes)`

Write raw bytes to PTY (Ctrl+C = `b"\x03"`, Ctrl+Z = `b"\x1a"`).

`terminate()`

Sends `SIGTERM` to child PID. Sets `_running = False`.

`is_running` (property)

Returns `bool` — whether process thread is active.

---

12. TEACH ENGINE — KNOWLEDGE BASE

File: `src/education/teach_engine.py`

`handle_teach_request(query) → dict`

Main API handler. Returns `{status, source, stdout}`.

`find_lesson(query) → dict | None`

Strips prefixes → checks LESSONS dict → checks KEYWORD_MAP → fuzzy check.

`format_lesson(lesson) → str`

Renders lesson dict into terminal-formatted text block with sections.

Built-in Lesson Topics (13 total)

Key	Title
`nmap`	NMAP — Network Mapper
`sqlmap`	SQLMap — Automatic SQL Injection
`nikto`	Nikto — Web Server Scanner
`gobuster`	Gobuster — Directory & DNS Brute Forcer
`hydra`	Hydra — Network Login Brute Forcer
`metasploit`	Metasploit Framework
`hashcat`	Hashcat — GPU Password Cracker
`aircrack`	Aircrack-ng — WiFi Security Toolkit
`nuclei`	Nuclei — Fast Vulnerability Scanner
`subfinder`	Subfinder — Subdomain Discovery
`sql injection`	SQL Injection (SQLi) Concept
`xss`	Cross-Site Scripting Concept
`meterpreter`	Meterpreter — Advanced Payload
`privilege escalation`	PrivEsc — Linux & Windows
`burp suite`	Burp Suite — Web App Proxy
`wireshark`	Wireshark — Packet Analyzer

Lesson Sections (per topic)

`title` · `tldr` · `what` · `how` · `phases` · `commands` · `flags` · `defense` · `tips` Optional: `modules`, `tools`, `wordlists`, `hash_modes`, `filters`, `key_exploits`

---

13. NATURAL LANGUAGE INTERFACE

File: `src/ai/natural_language.py — NaturalLanguageInterface`

`__init__(llm_router=None)`

Builds intent patterns + tool keywords. Creates `ToolExecutor` instance.

`parse_command(user_input) → ParsedIntent`

Full NLP pipeline: detect intent → extract target → select tools → extract params → calculate confidence. Returns `ParsedIntent(intent, target, tools, parameters, confidence, raw_command)`

`_detect_intent(text) → IntentType`

Regex match against patterns for: `SCAN, EXPLOIT, CRACK, ENUMERATE, ANALYZE, REPORT, HELP, UNKNOWN`

`_extract_target(text) → str | None`

Regex extraction for: domain names, IPv4 addresses, MD5/SHA hashes, URLs.

`_select_tools(text, intent) → List[str]`

Scores tools by keyword match frequency. Returns top 3. Falls back to intent defaults.

`_extract_parameters(text, intent, tools) → dict`

Extracts: `ports`, `timing`, `version_detection`, `os_detection`, `attack_type`, `wordlist`, `output_format`

`_calculate_confidence(intent, target, tools) → float`

Score: intent found (+0.3) + target found (+0.3) + tools found (+0.2). Max 1.0.

`async execute_command(user_input, mode="interactive") → dict`

Parses + executes. Mode `"interactive"` returns confirmation request. Mode `"yolo"` auto-runs.

`async _execute_tools(parsed) → dict`

Runs each tool via `_run_tool()`. Collects all results.

`async _run_tool(tool_name, target, params) → dict`

Calls `executor.run()`. Returns normalized result dict.

`_get_suggestions(user_input) → List[str]`

Returns 5 example command suggestions.

`get_help(topic=None) → str`

Returns help text. Topic-specific if provided.

---

14. AGENT ORCHESTRATOR

File: `src/ai/agents/orchestrator.py — AgentOrchestrator`

`__init__(llm_router=None, allow_remote=False)`

Instantiates: CVEIntelligenceAgent, BrowserAutomationAgent, ExploitGeneratorAgent, ToolExecutor.

`async autonomous_pentest(target, software_hints=None) → dict`

Full 5-phase autonomous pentest pipeline:

Phase	Method	Output
1. Intelligence	`_phase_intelligence()`	CVE count, critical count, exploitable list
2. Web Crawl	`_phase_web_crawl()`	Pages, forms, JS libs, missing headers
3. Exploit Planning	`_phase_exploit_planning()`	Exploit plans per CVE + web findings
4. Execution	`_phase_execution()`	Tool run results, findings count
5. Report	`_build_report()`	Full Markdown pentest report

`async _phase_intelligence(target, software_hints) → dict`

Runs `cve_agent.analyze_target()`. Returns CVE stats.

`async _phase_web_crawl(target) → dict`

Runs `browser_agent.crawl_and_analyze()`. Returns web findings.

`async _phase_exploit_planning(intel, web, target) → dict`

Generates plans from CVEs + web findings via `exploit_agent`.

`async _phase_execution(exploit, target) → dict`

Walks `tool_chain` in each plan. Runs tools via `ToolExecutor`. Aggregates findings.

`_build_report(results) → str`

Static method. Builds full Markdown report from all phase results.

`get_agent_status() → dict`

Returns status of all 4 sub-agents: `{cve_intelligence, browser_automation, exploit_generator, tool_executor}`

---

15. SOC BLUE TEAM — `/api/soc` ACTIONS

POST `{"action": "<action_name>"}` to `/api/soc`

Action	Command

`failed-logins`	`grep 'Failed password' /var/log/auth.log` + journald SSH audit
`active-conns`	`ss -tulpn && netstat -an
`open-ports`	`ss -tulpn
`running-procs`	`ps aux --sort=-%cpu
`log-tail`	`tail -n 50 /var/log/auth.log /var/log/syslog`
`who-online`	`who && last
`sudo-log`	`grep sudo /var/log/auth.log
`firewall-rules`	`iptables -L -n -v` or `ufw status verbose`
`cron-jobs`	`crontab -l && ls -la /etc/cron*`
`dns-cache`	`resolvectl statistics && cat /etc/hosts`
`volatility`	`volatility3 --info`

---

16. PENTEST TOOL QUICK REFERENCE

NMAP Common Flags

`-sS` SYN scan · `-sV` service version · `-sC` default scripts · `-A` aggressive · `-p-` all ports · `-F` fast · `-O` OS detection · `-oN` save file · `-T4` faster timing · `--script vuln` vulnerability scripts · `--min-rate 5000` speed boost

SQLMap Common Flags

`-u` URL · `--data` POST data · `--dbs` list databases · `--tables` list tables · `--dump` dump table · `--dump-all` dump everything · `--os-shell` OS shell · `--cookie` session cookie · `-r` request file · `--batch` non-interactive · `--tamper` WAF bypass · `--level` test depth (1-5) · `--risk` aggressiveness (1-3)

Gobuster Wordlists

- Quick: `/usr/share/wordlists/dirb/common.txt`
- Medium: `/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt`
- DNS: `/usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt`

Hashcat Hash Modes

`0` =MD5 · `100` =SHA1 · `1000` =NTLM · `1400` =SHA256 · `1800` =sha512crypt · `3200` =bcrypt · `5600` =NetNTLMv2 · `13100` =Kerberoast · `22000` =WPA2

Metasploit Key Commands

`search type:exploit` · `use <module>` · `set RHOSTS <ip>` · `set LHOST <your-ip>` · `set PAYLOAD windows/x64/meterpreter/reverse_tcp` · `run / exploit` · `sessions -l` · `sessions -i 1` · `background`

Meterpreter Key Commands

`sysinfo` · `getuid` · `getsystem` · `hashdump` · `ps` · `migrate <pid>` · `upload` · `download` · `screenshot` · `keyscan_start` · `keyscan_dump` · `shell` · `run post/multi/recon/local_exploit_suggester`

---

17. PAYLOAD STUDIO VALIDATORS

`_validate_ducky(code) → dict`

Returns `{status, valid, errors:[{line, msg}], line_count}`

Valid DuckyScript commands: `REM STRING STRINGLN DELAY GUI CTRL ALT SHIFT ENTER BACKSPACE TAB ESCAPE DELETE UPARROW DOWNARROW LEFTARROW RIGHTARROW HOME END PAGEUP PAGEDOWN CAPSLOCK PRINTSCREEN MENU SPACE WAIT_FOR_BUTTON_PRESS HOLD RELEASE REPEAT VAR IF WHILE DEFINE DEFAULT_DELAY LOCALE F1–F12`

`_explain_line(line) → dict`

Calls `payload_engine.get_line_explanation(line)`. Returns `{status, explanation}`.

`_get_suggestions(code, platform, last_line) → dict`

Calls `payload_engine.get_suggestions()`. Returns `{status, suggestions:[]}`.

ERR0RS SERVER INTERNALS

`start_server()`

Starts HTTP server in daemon thread on `127.0.0.1:8765`. Opens browser.

`start_ws_server()`

Starts WebSocket server in daemon thread on `127.0.0.1:8766`.

`ws_terminal_handler(websocket)`

Async WebSocket handler. Manages session, routes messages, spawns LiveProcess, handles teach mode.

`check_ollama()`

Probes `localhost:11434/api/tags`. Reports model list or install instructions.

`check_ws_deps()`

Checks if `websockets` library is installed.

`_uptime() → str`

Reads `/proc/uptime`. Returns `HH:MM:SS` string.

`_platform() → str`

Reads `/proc/cpuinfo` for Raspberry Pi detection.

Environment Variables

Variable	Default	Description
`ERRZ_HOST`	`127.0.0.1`	Bind address
`ERRZ_PORT`	`8765`	HTTP port
`ERRZ_WS_PORT`	`8766`	WebSocket port

Generated for ERR0RS Ultimate v3.0 | H:\ERR0RS-Ultimate | Stay legal. Stay safe.